

Final Group Assignment — Architectural Evolution of nopCommerce

Evolving a Known Platform Under Real Architectural Pressure

Context

This assignment asks you to evolve an existing production-grade system under explicit architectural pressure.

The base system for the assignment is **nopCommerce**:

- Repository: <https://github.com/nopSolutions/nopCommerce>
- Language: C# / ASP.NET Core
- Baseline architecture: modular monolith with layered / onion-style structure

nopCommerce is already capable software. That is intentional.

Your task is not to rewrite it from scratch or decorate it with fashionable technologies. Your task is to decide how it should evolve when the business around it becomes more demanding.

This is a **group assignment** for teams of **up to 4 students**.

Goal

Your team must:

1. analyse the current architecture of nopCommerce
2. choose one strategic scenario
3. define and justify an architectural evolution
4. implement a focused subset of that evolution
5. demonstrate how the evolved system behaves under both normal and pressured conditions

This is not a feature-only assignment.

The main evaluation target is the quality of your architectural judgment, the traceability of your decisions, and the credibility of your demonstration under pressure.

Assignment Spirit

You are expected to produce a **working architectural demonstration environment**.

That means your final result must make it possible to see:

- the chosen business context
- the surrounding systems relevant to that context
- the pressure points affecting the architecture
- the architectural response to those pressures
- the recovery or resolution behavior after degradation, delay, inconsistency, or failure

The final outcome should not be only:

- diagrams without runtime evidence
- code without architectural rationale
- a happy-path demo that never exercises the hard part of the design

The point is to show an architectural evolution of nopCommerce in a believable operating environment.

Scenario Choice

Choose **one** of the following scenarios.

The named surrounding systems are part of the assignment context. You may replace any named project with **or similar** if you can justify that the replacement creates the same architectural pressure.

You do **not** need to reproduce the full depth of every surrounding system. You do need to represent enough of the ecosystem to make the architectural problem real.

Any surrounding system may be delivered through a **surrogate simulator, stub, or mock service** if that choice preserves the pressure, behavior, and architectural consequences of the scenario.

Scenario A — Federated Commerce After Acquisitions

Northstar Living Group began as a single online retailer. Over time, growth came through acquisition: several specialist brands now sit under the same corporate umbrella, but each one still operates with its own catalog logic, pricing habits, inventory assumptions, and operational rhythm.

Leadership wants one stronger digital foundation without destroying local autonomy.

The challenge is not only deciding what should be shared and what should remain local. The challenge is also deciding **how to evolve toward that target** without either fragmenting into disconnected platforms or locking every business unit into one rigid model.

Strategic Goals

1. Create a shared digital foundation across the group.
2. Preserve meaningful local autonomy for each business unit.
3. Allow the platform to evolve without cross-unit instability and uncontrolled coupling.

Surrounding Systems

Use some of the following systems, or similar, to make the scenario real:

- [Keycloak](#) or [authentik](#) for shared identity, organization, and delegated access
- [ERPNext](#) or [Odoo Community](#) for unit-specific ERP / back-office behavior
- [Akeneo PIM Community Edition](#) or similar PIM for catalog enrichment and publishing
- [OpenBoxes](#) or similar inventory / supply-chain tooling
- [EspoCRM](#) or similar CRM for shared customer operations
- [OpenSearch](#) or [Meilisearch](#) for search and discovery pressure
- [RabbitMQ](#) or [Apache Kafka](#) for synchronization and delayed consistency pressure

Mandatory Demonstration Use Cases

Your final demonstration must include both:

1. A **shared customer experience across federated units**
 - for example, discovery, browsing, or customer-facing interaction that spans at least two business units
2. A **business-unit-specific order, stock, or fulfillment path**
 - showing that local operational differences can exist without collapsing into one tangled system

Mandatory Pressure Point

You must also demonstrate:

- one business unit suffering a degraded local subsystem, integration, or policy path **without taking down the whole group experience**

The demo must show:

- how the degradation becomes visible
 - how the architecture contains or routes around it
 - how the affected path recovers or reconciles
-

Scenario B — Regulated / Controlled Commerce

Helix Industrial Supply began as a conventional online seller of industrial equipment and technical components. It now wants to expand further into regulated and controlled offerings, where documentation, geography, certification status, approval rules, and downstream handling obligations matter.

Commerce is no longer only about products and customers. It is also about policy, evidence, auditability, change over time, and safe behavior when verification or approval dependencies are imperfect.

The challenge is not "add compliance checks." The challenge is to design for a world where rules evolve, dependencies degrade, and the business must still behave safely and explainably.

Strategic Goals

1. Support regulated or controlled product flows without reducing the problem to checkout validation.
2. Preserve traceability, explanation, and defensible decision-making.
3. Remain operational under changing policies and degraded verification dependencies.

Surrounding Systems

Use some of the following systems, or similar, to make the scenario real:

- [Keycloak](#) or [authentik](#) for identity, trusted users, groups, and approvals
- [OpenFGA](#) or similar authorization system for delegated or relationship-based access
- [Open Policy Agent](#) or similar policy engine for evolving rule evaluation
- [Paperless-ngx](#) or [Mayan EDMS](#) for certificates, declarations, and supporting evidence
- [ERPNext](#) or [Odoo Community](#) for traceable operational state
- [RabbitMQ](#) or [Apache Kafka](#) for approvals, retries, and delayed external confirmation
- [WireMock](#) or similar API simulator for verifier / regulator / external dependency behavior
- [OpenSearch](#) or similar search / audit visibility tooling

Mandatory Demonstration Use Cases

Your final demonstration must include both:

1. A **controlled product purchase with visible policy evaluation**
 - where policy, authorization, verification, or supporting evidence materially affects the flow
2. A **policy-sensitive or document-sensitive approval flow**
 - where a product, customer, or order cannot proceed normally without extra checks, evidence, or approval

Mandatory Pressure Point

You must also demonstrate:

- a verifier, approval dependency, or regulation-facing integration becoming **slow, unavailable, or contradictory** while the system remains responsible for safe behavior

The demo must show:

- what the system allows, delays, blocks, or records during the degraded period
 - how traceability and explanation are preserved
 - how the system recovers once the dependency or policy state stabilizes
-

Scenario C — Omnichannel Commerce Core

VerdeMart Retail began with nopCommerce as a web storefront for a growing retail business. The company now wants a more unified commerce experience across channels: web, store operations, warehouse execution, shipping, support, and customer visibility all need to work together.

nopCommerce is no longer only the online shop. It is becoming the **commerce core** in a wider enterprise ecosystem.

The challenge is not merely connecting more systems. The challenge is deciding how the commerce core should behave when surrounding systems disagree, lag, degrade, or recover.

Strategic Goals

1. Make nopCommerce act as a reliable commerce core rather than an isolated storefront.
2. Improve cross-channel visibility of stock, order state, and fulfillment progress.
3. Remain useful when surrounding operational systems are stale, delayed, or temporarily unavailable.

Surrounding Systems

Use some of the following systems, or similar, to make the scenario real:

- [ERPNext](#) or [Odoo Community](#) for ERP / back-office pressure
- [OpenBoxes](#) or similar WMS / inventory platform
- [Open Source Point of Sale](#) or similar POS system
- [EspoCRM](#) or similar CRM
- [OpenSearch](#) or [Meilisearch](#) for search freshness and discovery pressure
- [RabbitMQ](#) or [Apache Kafka](#) for propagation and async coordination
- [WireMock](#) or similar API simulator for shipping or carrier behavior
- [Keycloak](#) or [authentik](#) for identity across channels

Mandatory Demonstration Use Cases

Your final demonstration must include both:

1. A **buy-online / fulfill-through-another-channel** flow
 - for example, an order placed in nopCommerce that must be reflected in ERP, warehouse, store, or shipping operations
2. A **cross-channel stock or order-state visibility** flow
 - where a change outside nopCommerce becomes visible back in the commerce experience

Mandatory Pressure Point

You must also demonstrate:

- a surrounding operational system becoming **delayed, stale, unavailable, or contradictory** while the commerce core must remain useful

The demo must show:

- how the degradation or inconsistency becomes visible
- how the commerce core behaves during the degraded period
- how retries, coordination, or reconciliation lead to recovery

What You Must Do

Your work must include both architectural design and implementation.

Required Architectural Work

You must:

1. Analyse the current nopCommerce architecture and identify the parts most relevant to your scenario.
2. Define the main business drivers and architectural drivers.
3. Model the relevant domains and bounded contexts.
4. Write **3 to 5 quality attribute scenarios** that drive your design.
5. Choose and apply **at least one software architecture design framework**:
 - ADD
 - ACDM
 - ADM / TOGAF
6. Explain why that framework fits your scenario.
7. Produce and justify the target architecture.
8. Document the **architectural evolution path** from current state to target state.
9. Record major decisions using ADRs, including rejected alternatives.
10. Implement a focused subset of the evolution and demonstrate it under pressure.

Required Technical Constraints

Your solution must satisfy all of the following:

1. Introduce at least **one asynchronous workflow**.
2. Make at least **one explicit reliability decision** such as:
 - idempotency
 - outbox
 - retry policy
 - compensation
 - dead-letter handling
 - circuit breaker
3. Represent at least **two surrounding systems or equivalent external capabilities** from the scenario ecosystem.
4. Include at least **one independently deployable subsystem or extracted service**, if your design justifies it.
5. Do **not** use a shared database as a shortcut across extracted service boundaries.
6. Justify what remains inside the monolith and why.
7. Show evidence of performance, scalability, resilience, operability, or traceability improvements relevant to the scenario.

Creative Freedom

You are encouraged to think creatively on top of the base scenario.

That means you may:

- extend the scenario with additional constraints or stakeholders

- choose different open-source tools than the named ones, if they create the same pressure
- add supporting capabilities that deepen the architectural challenge
- make selective scope cuts, if you justify them honestly

However, creative additions must **strengthen** the architectural story, not obscure it.

Your submission must still make the original scenario, mandatory use cases, and mandatory pressure point clearly visible.

Scope Constraint

You are expected to make **selective** architectural changes.

This is not a "rewrite nopCommerce into microservices" assignment.

A smaller, defensible architecture is better than an inflated design that is only convincing in diagrams.

Documentation and Traceability

This is a software architecture assignment. Documentation is part of the work.

Your repository should make the architectural evolution visible through:

- an architecture report or equivalent structured documentation
- current-state analysis
- target-state architecture
- evolution roadmap
- bounded-context view
- runtime interaction diagrams where relevant
- quality attribute scenarios
- ADRs
- setup and run instructions
- evidence pack: measurements, experiments, failure scenarios, and known limitations

A reviewer should be able to move from:

- scenario and drivers
- to quality attributes
- to framework and decisions
- to implementation choices

- to evidence and demonstration
-

What Will Not Score Well

The following will be treated as weak submissions:

- large amounts of generated code with little architectural justification
 - feature work that does not produce a meaningful architectural problem
 - diagrams that do not correspond to real runtime behaviour
 - service decomposition without clear boundaries or ownership
 - architecture decisions not linked to business drivers or quality attributes
 - demos that show only happy paths
 - claims of scalability, resilience, or operability without evidence
 - implementations that rely on hidden coupling across boundaries
 - surrounding systems that are named but do not materially affect the architecture
-

Part 1 — Architecture Checkpoint

Objective

By the first presentation, your team must already have a meaningful portion of the architecture decided.

This is a design checkpoint, not a progress update.

What You Must Present

Your Part 1 presentation must include:

1. **Scenario choice**

- Which scenario you selected
- Why it matters from a business perspective

2. **Current-state analysis**

- How nopCommerce currently supports or conflicts with your scenario
- The architectural seams and pressure points you identified

3. **Domain and boundary model**

- Relevant subdomains
- Bounded contexts
- Ownership of major responsibilities

4. **Quality attribute scenarios**

- 3 to 5 concrete scenarios that will drive the design

5. **Chosen framework**

- Which method you selected (ADD, ACDM, or ADM)
- Why it fits your case

6. **Target architecture**

- Main components or services
- Data ownership
- Synchronous and asynchronous interactions
- Cross-cutting concerns that shape the design

7. **Architectural decisions**

- At least **3 ADRs**
- Including rejected alternatives

8. **Risk and validation plan**

- Main architectural risks
- What you will build or measure to de-risk them

9. **Evolution roadmap**

- What you will implement next
- In what order
- What must coexist during the transition

10. **Feasibility spike**

- One small but concrete technical spike on the riskiest part of the design

Suggested Presentation Format

- 10 to 12 minutes presentation

- 5 minutes technical discussion

What We Will Look For

- Are the main decisions already made?
 - Are the boundaries credible?
 - Is the design traceable to explicit drivers?
 - Does the plan show control of scope and risk?
 - Is the evolution path believable?
 - Is the team prepared to defend rejected alternatives?
-

Part 2 — Final Delivery

Objective

Implement and demonstrate the architectural evolution you proposed.

What You Must Deliver

1. **Working implementation**

- A runnable repository
- Clear setup and execution instructions

2. **Architecture report**

- Short and focused
- Enough to explain the rationale, the evolution path, and the final design

3. **ADR set**

- Updated to reflect final decisions

4. **Evidence pack**

- Measurements
- Test scenarios
- Experiment results
- Known limitations

5. **Final presentation and demo**

- Live demonstration of the evolved system

- Defense of design decisions and tradeoffs

What You Must Demonstrate

Your final presentation must show:

1. The required use cases for your chosen scenario working end to end.
 2. The mandatory pressure point for your chosen scenario being exercised.
 3. The architectural changes made relative to the baseline nopCommerce system.
 4. The behavior of the system during degradation, inconsistency, delay, or failure.
 5. The recovery or resolution path after the pressure condition is removed or stabilized.
 6. Evidence that the architecture improved the target qualities.
 7. The main limits of your design.
-

Required Artifacts

Your repository must contain:

- source code
- build and run instructions
- architectural diagrams
- bounded-context view
- quality attribute scenarios
- chosen framework and how you applied it
- ADRs
- experiment or measurement evidence
- any configuration notes required to reproduce the demo

Use a structure that makes traceability clear.

Evaluation

Overall Weighting

- **Part 1: Architecture Checkpoint** — 20%
- **Part 2: Implementation and Demonstration** — 80%

Part 1 Evaluation

Part 1 will be evaluated on:

1. Quality of current-state analysis.
2. Quality of domain and boundary modeling.
3. Strength of the quality attribute scenarios.
4. Methodology choice and its application.
5. Quality of the architectural decisions.
6. Quality of rejected alternatives.
7. Risk awareness and validation plan.
8. Realism of the implementation plan.
9. Technical defense.

Part 2 Evaluation

Part 2 will be evaluated on:

1. Architectural coherence between design and implementation.
2. Quality of the selective architectural evolution.
3. Correct handling of reliability and consistency concerns.
4. Performance, scalability, resilience, operability, or traceability evidence.
5. Quality of security and cross-cutting decisions.
6. Clarity and credibility of the demonstration.
7. Traceability from requirements to implementation.
8. Technical defense.

Use of AI Tools

You may use AI-assisted tools, including code generation and analysis tools.

That does not reduce your responsibility for the work.

You are expected to:

- understand all code and design decisions you submit
- defend the architectural tradeoffs in your own words
- explain rejected alternatives
- justify your evidence and measurements

Generated code without a coherent design rationale will not score well.

Practical Guidance

A strong project for this assignment usually has:

- one clear architectural problem
- a small number of important decisions
- strong traceability
- focused implementation
- a visible pressure condition
- a credible recovery story

A weak project usually tries to impress by increasing the number of technologies rather than the quality of the design.

Prefer depth over breadth.

Final Reminder

The assignment is not to "make nopCommerce distributed."

The assignment is to make and defend a set of architectural decisions that evolve nopCommerce appropriately for one demanding scenario, in a runnable environment where the architecture can be seen under pressure.